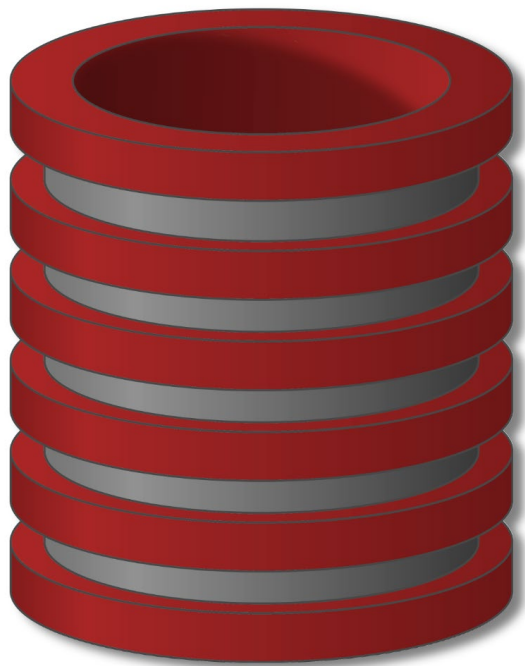




Introduction to advanced topics

NoSQL Systems
Data Systems
DB research topics

NoSQL Systems

Overview

NoSQL: The Name

- “SQL” = Traditional relational DBMS
- Recognition over past decade or so:
Not every data management/analysis problem is best solved using a traditional relational DBMS
- “NoSQL” = “No SQL” =
Not using traditional relational DBMS
- “No SQL” $\neq$ Don't use SQL language

NoSQL: The Name

- “SQL” = Traditional relational DBMS
- Recognition over past decade or so:
Not every data management/analysis problem is best solved using a traditional relational DBMS
- “NoSQL” = “No SQL” =
Not using traditional relational DBMS
- “No SQL” $\neq$ Don't use SQL language
- ✱ “NoSQL” = “Not Only SQL”

Not every data management/analysis problem is best solved using a traditional DBMS

Database Management System (DBMS) provides....

... efficient, reliable, convenient, and safe multi-user storage of and access to massive amounts of persistent data.

NoSQL Systems

Alternative to traditional relational DBMS

- + Flexible schema
- + Quicker/cheaper to set up
- + Massive scalability
- + Relaxed consistency → higher performance & availability
- No declarative query language → more programming
- Relaxed consistency → fewer guarantees

Example #1: Web log analysis

Each record: UserID, URL, timestamp, additional-info

Task: Load into database system

Example #1: Web log analysis

Each record: UserID, URL, timestamp, additional-info

Task: Find all records for...

- Given UserID
- Given URL
- Given timestamp
- Certain construct appearing in additional-info

Example #1: Web log analysis

Each record: UserID, URL, timestamp, additional-info

Separate records: UserID, name, age, gender, ...

Task: Find average age of user accessing given URL

Example #2: Wikipedia pages

Large collection of documents

Combination of structured and unstructured data

Task: Retrieve introductory paragraph of all pages about U.S. presidents before 1900

NoSQL Systems

Several incarnations

- MapReduce framework
- Key-value stores
- Document stores
- Graph database systems

Overview of Data Systems

Core questions related to a data system

1. How to **interact** with a data system?
2. How does a data system **model** the data?
3. How does a data system **store** the data?
4. How does a data system **index** the data?
5. How does a data system answer a **query**?
6. How does a data system support **analytics**?

1. How to interact with a data system?

- **Code-Centric Approach**

- **Imperative** programming (i.e., describe how to do things step-by-step)
- Spark (Scala/Java) for batch processing
- Ad hoc code (Python) for exploration

- **Query-Centric Approach**

- **Declarative** programming (i.e., declare the things you want)
- **SQL (Structured Query Language)** for both interactive exploration and batch processing

2. How does a data system **model** the data?

- **Relational model**
- **Key-value model**
 - Set of key-value pairs

KEY1	→	Value1
KEY2	→	Value2
KEY3	→	Value3
KEY4	→	Value4
KEY5	→	Value1
KEY6	→	Value3

(a)

Key	Value
user 1: employee	{65,865,9634}
user2: employee	{34,85,76,94}
user3: employee	{desg:manager, branchcode: 345}
user4: employee	{487,236}
user5: employee	{78,456,35}
user6: employee	{ name: mark, empid:346}

(b)

a) Key-value store (KVS) data model, (b) Data in KVS

Key-Value Stores

Extremely simple interface

- **Data model:** (key, value) pairs
- **Operations:** Insert(key,value), Fetch(key), Update(key), Delete(key)

Implementation: efficiency, scalability, fault-tolerance

- Records distributed to nodes based on key
- Replication
- Single-record transactions, “eventual consistency”

Key-Value Stores

Extremely simple interface

- Data model: (key, value) pairs
- Operations: Insert(key,value), Fetch(key), Update(key), Delete(key)
- Some allow (non-uniform) columns within value
- Some allow Fetch on range of keys

Example systems

- Google BigTable, Amazon Dynamo, Cassandra, Voldemort, HBase, ...

2. How does a data system **model** the data?

- **Other data models**

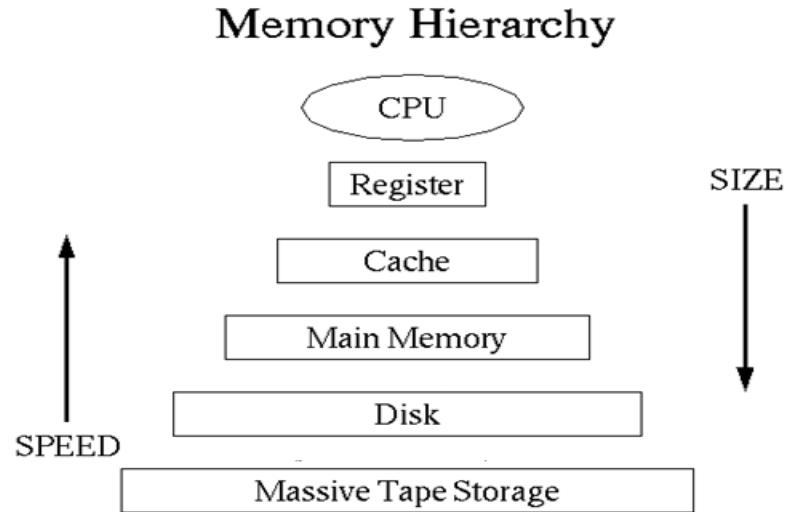
- Matrix, tensor, and spreadsheets
 - HTML/XML/JSON
 - Geographic data model (vector/raster files)
 - ...
- For details, refer to the Wikipedia page of “data model” at https://en.wikipedia.org/wiki/Data_model

2. How does a data system **model** the data?

- **Relational model**
- Key-value model
- Graph model
- Matrix, tensor, and spreadsheets
- HTML/XML/JSON
- Geographic data model (vector/raster files)
- ...

3. How does a data system **store** the data?

- **Memory Hierarchy**

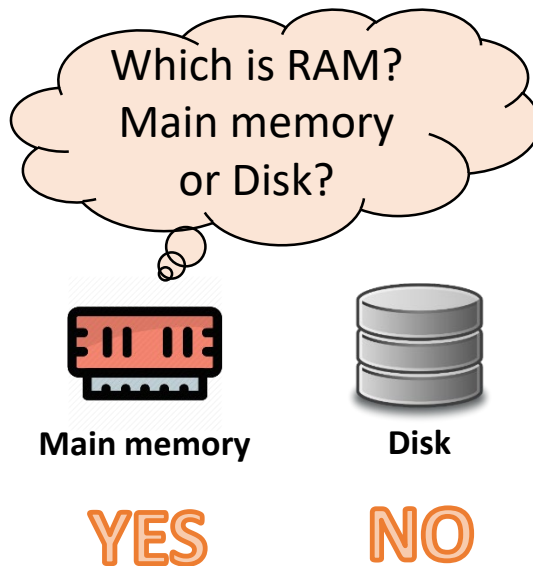


3. How does a data system **store** the data?

Random access memory (RAM):

Irrespective of the physical locations of data, the time to access the data is the same

1. <https://cdn3.iconfinder.com/data/icons/computer-and-hardware-2-1/34/114-512.png>
2. <https://www.nicepng.com/maxp/u2q8u2o0i1w7o0u2/>

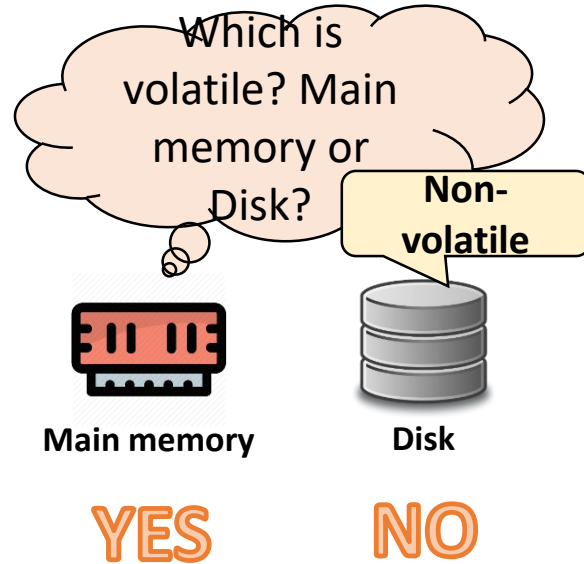


3. How does a data system **store** the data?

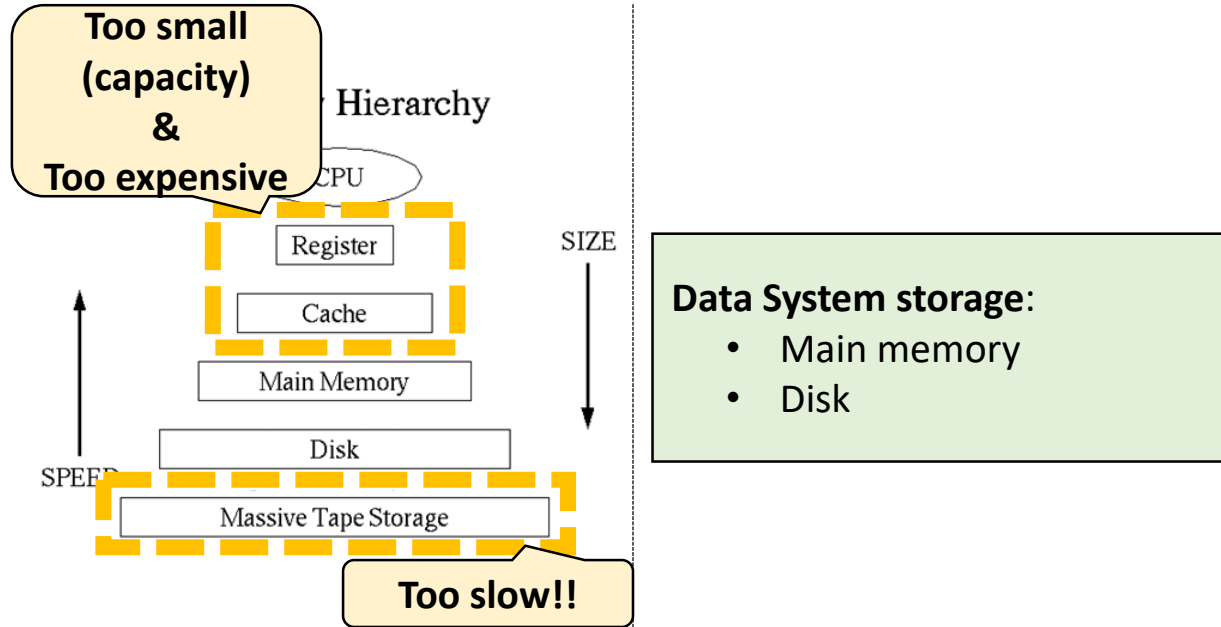
Volatile storage:

Storage would be lost
when the power goes off.

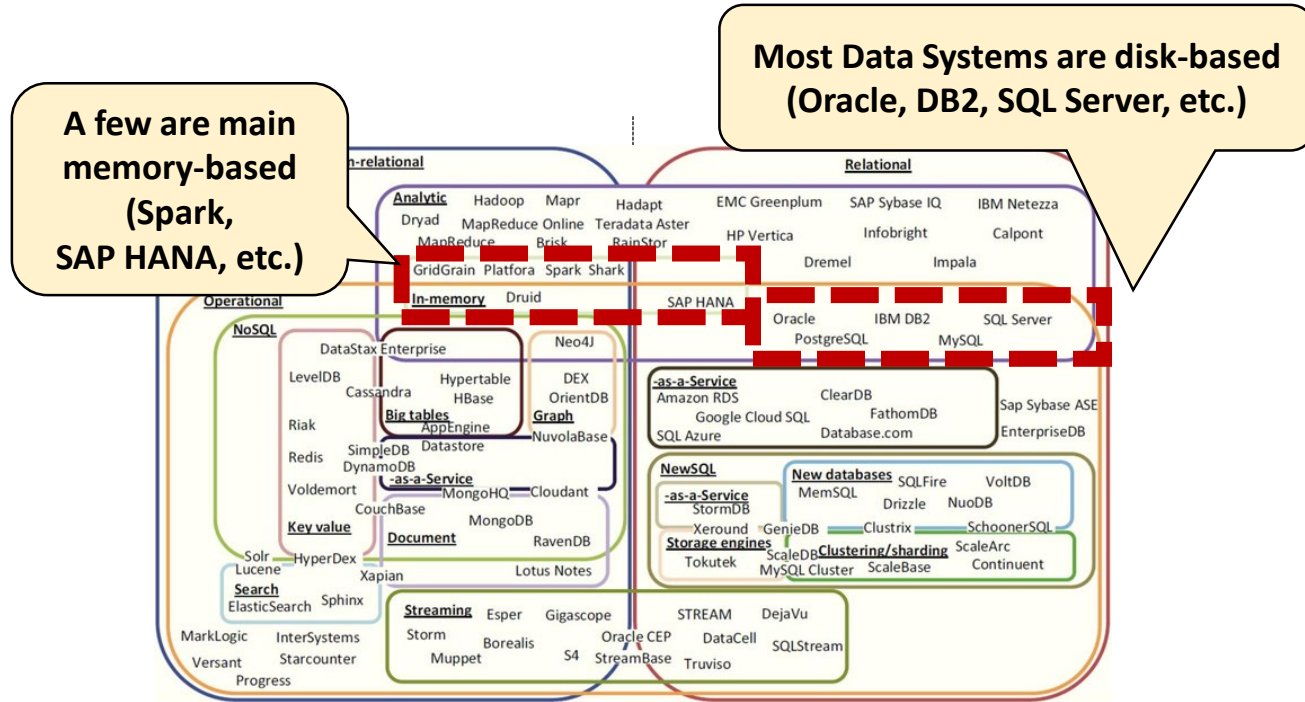
1. <https://cdn3.iconfinder.com/data/icons/computer-and-hardware-2-1/34/114-512.png>
2. <https://www.nicepng.com/maxp/u2q8u2o0i1w7o0u2/>



3. How does a data system **store** the data?

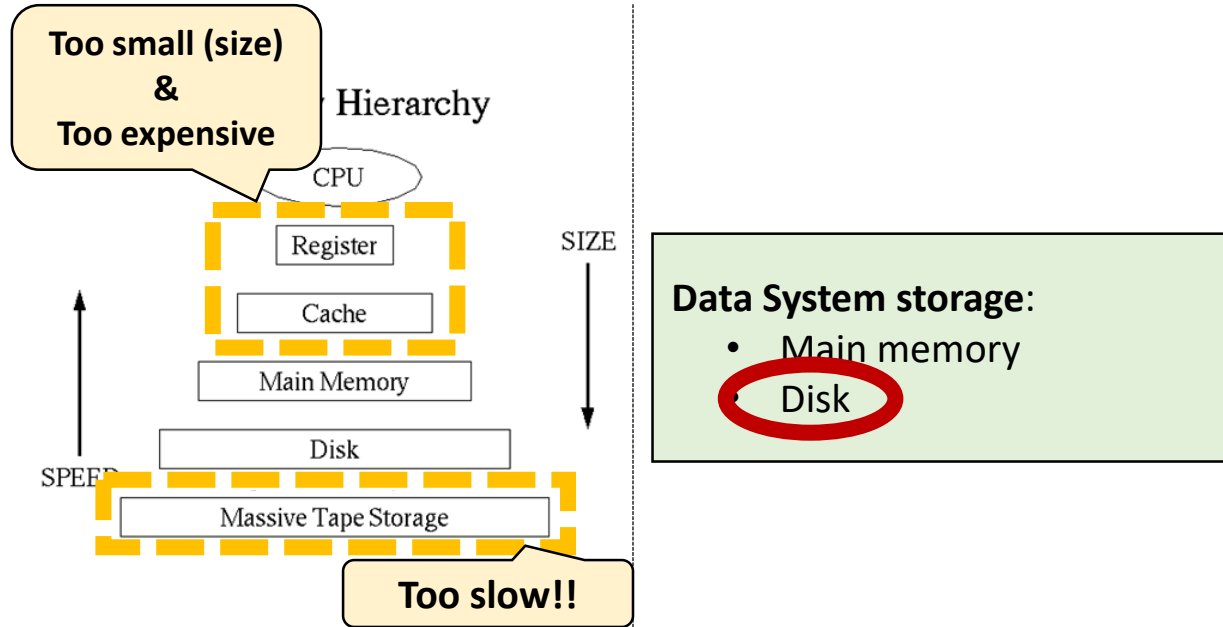


3. How does a data system **store** the data?



https://www.researchgate.net/figure/Landscape-and-categorization-of-the-high-variety-of-existing-database-systems-18_fig2_303562879

3. How does a data system **store** the data?



3. How does a data system **store** the data?

- **Row-oriented storage** and **Column-oriented storage**

Item	Color	Size	Number
[Orange squiggle]			
[Orange squiggle]			

Row-oriented storage

Item	Color	Size	Number
[Orange squiggle]	[Orange squiggle]	[Orange squiggle]	[Orange squiggle]
[Orange squiggle]	[Orange squiggle]	[Orange squiggle]	[Orange squiggle]

Column-oriented storage (Case 1)

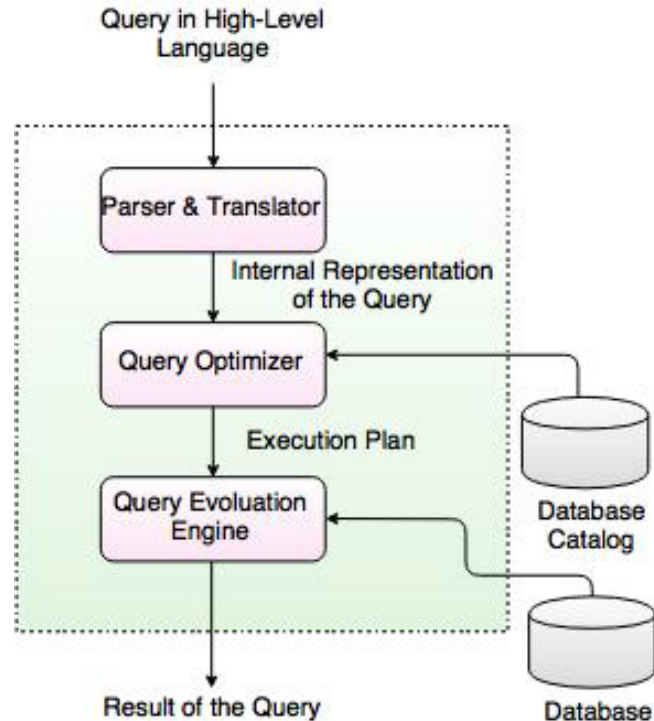
Item	Size	Color	Number
[Orange squiggle]	[Orange squiggle]	[Orange squiggle]	
[Orange squiggle]	[Orange squiggle]	[Orange squiggle]	

Column-oriented storage (Case 2)

4. How does a data system **index** the data?

- Conventional indexes
- B+ Tree
- Hash Index
- Multiple Key index

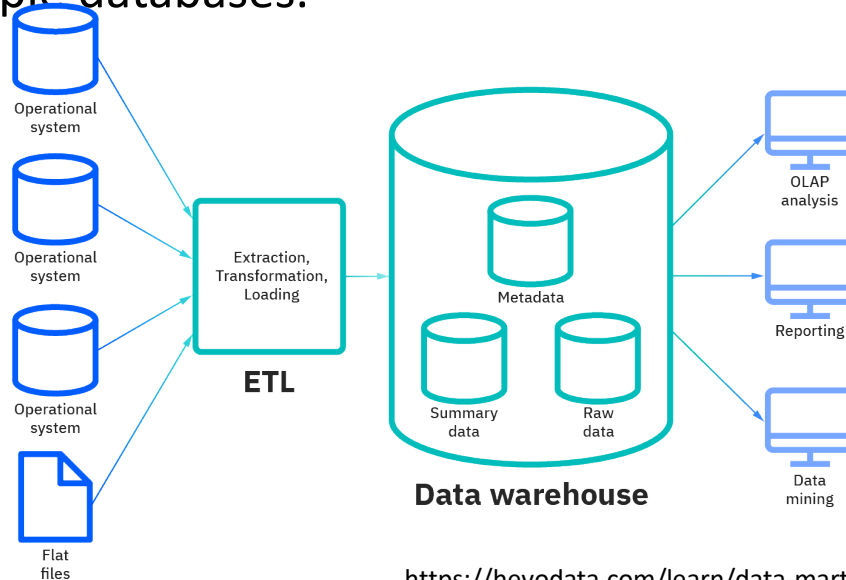
5. How does a data system answer a query?



Query Processing

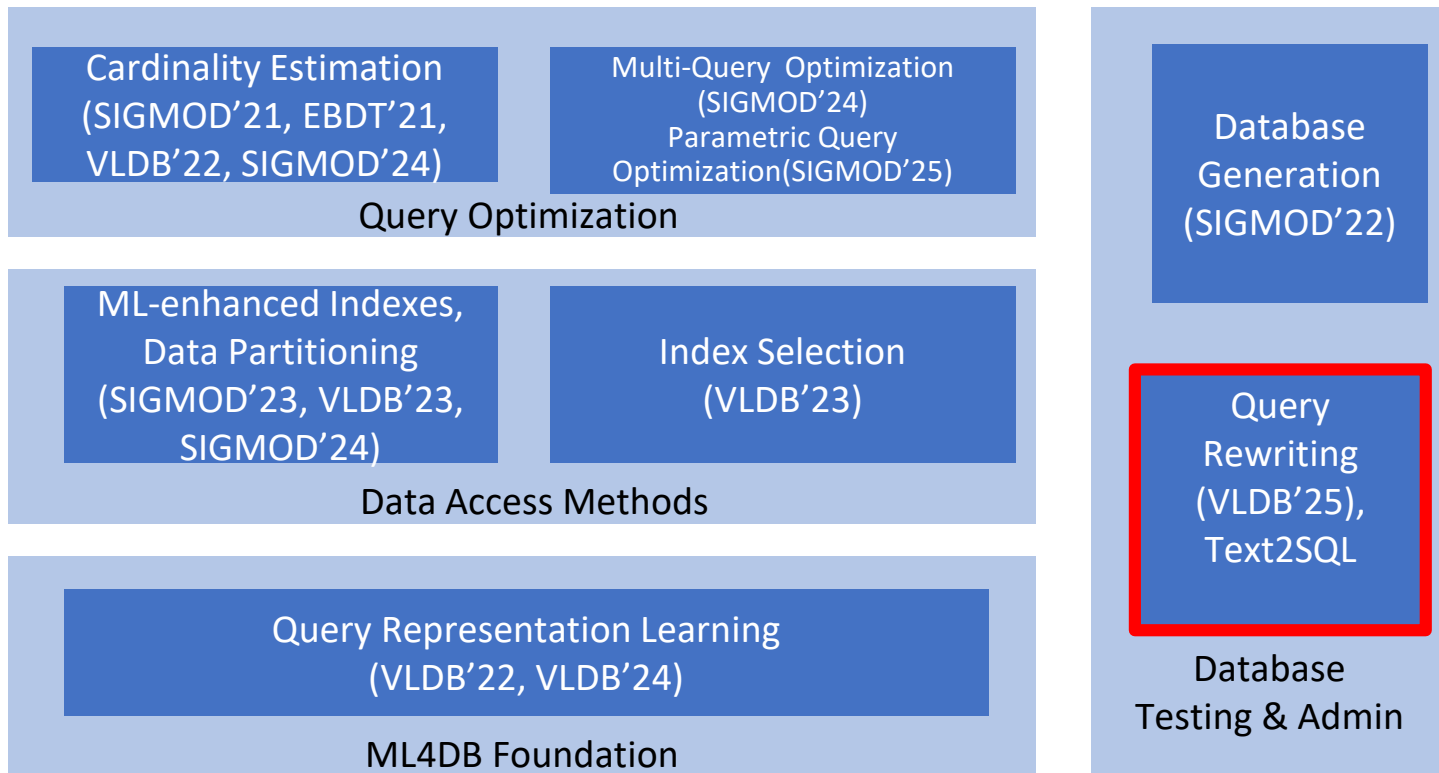
6. How does a data system support **analytics**?

- A **Data Warehouse** is read-focused and is optimized for **analytical tasks (OLAP, Reporting, Data Mining)**. A typical Data Warehouse can contain data from multiple databases.



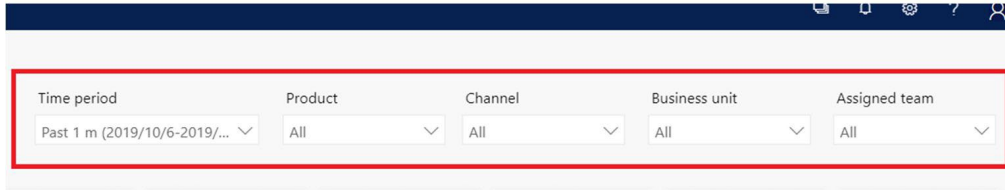
DB research

Overview of our Research on AI for Database (AI4DB)



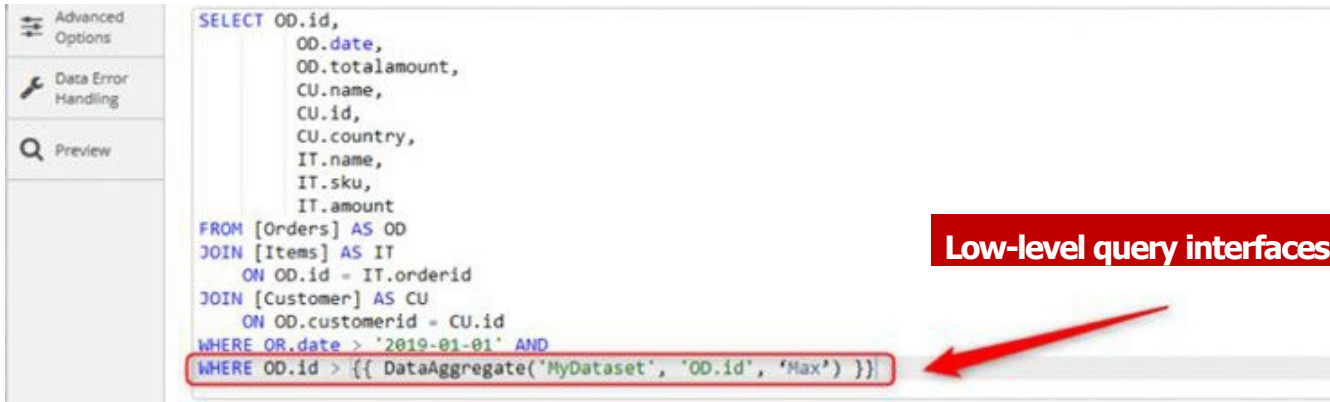
Why Text-to-SQL Systems?

Data Query Interfaces are user-unfriendly



A screenshot of a web-based query interface. It features a horizontal row of five dropdown menus. The first menu is labeled 'Time period' and shows 'Past 1 m (2019/10/6-2019/...'. The other four menus are labeled 'Product', 'Channel', 'Business unit', and 'Assigned team', each showing 'All'. A red rectangular box highlights this entire row of filters.

Form-based interfaces have limited query capabilities



A screenshot of a SQL query editor. On the left is a sidebar with icons for 'Advanced Options', 'Data Error Handling', and 'Preview'. The main area contains a SQL query. The query is as follows:

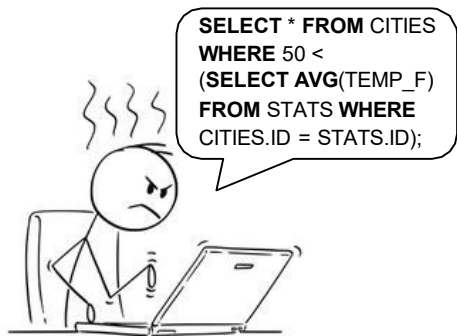
```
SELECT OD.id,
       OD.date,
       OD.totalamount,
       CU.name,
       CU.id,
       CU.country,
       IT.name,
       IT.sku,
       IT.amount
FROM [Orders] AS OD
JOIN [Items] AS IT
    ON OD.id = IT.orderid
JOIN [Customer] AS CU
    ON OD.customerid = CU.id
WHERE OD.date > '2019-01-01' AND
WHERE OD.id > [{ DataAggregate('MyDataset', 'OD.id', 'Max') }]
```

The last line of the query, `WHERE OD.id > [{ DataAggregate('MyDataset', 'OD.id', 'Max') }]`, is enclosed in a red rectangular box. A red arrow points from the right towards this box.

Low-level query interfaces are intended for programmers

Why Text-to-SQL Systems?

- Data **volume and complexity** make it difficult to query data
- Database query interfaces are notoriously **user-UNFRIENDLY**



What is **data democratisation**?

- **Empower everyone** to access, use, understand and derive value from data
- Lift the **technical barriers** that impede access to data and **eliminate dependency** to IT experts
- Design tools that are aimed for the **casual user**
- An organization-wide cultural stance

Why Text-to-SQL Systems?

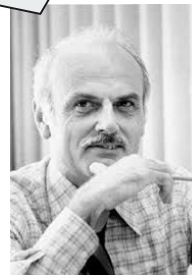
```
SELECT * FROM CITIES  
WHERE 50 <  
(SELECT AVG(TEMP_F)  
FROM STATS WHERE  
CITIES.ID = STATS.ID);
```

Which cities have
year-round average
temperature above
50 degrees?



To satisfy the needs of casual users of databases,
we must break through the barriers that presently prevent
these users from freely **employing their native languages**

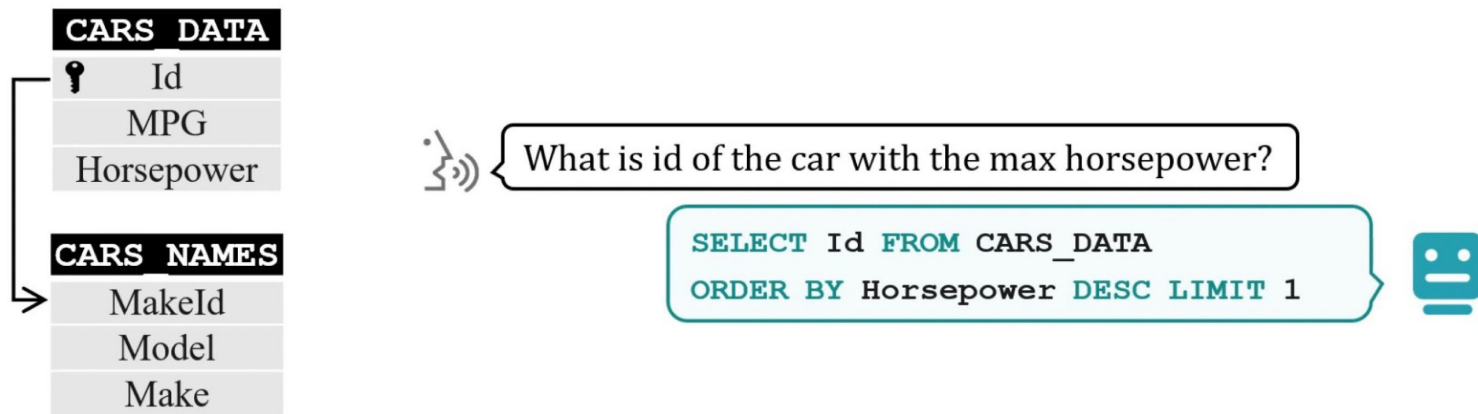
Ted Codd (circa: 1974)



Expressing queries in natural language can open up data access to everyone

Problem Definition

- Text-to-SQL aims at converting **natural language questions** into **executable SQL queries**. The problem has garnered increasing attention, as it can assist end users in efficiently extracting vital information from databases without need for the technical background.



Challenges: From the NL side

- Complexity of NL

- Ambiguity
- References - Schema Linking
- Inferences
- Vocabulary Gap

- User Mistakes

- Spelling mistakes
- Syntactical/Grammatical mistakes

"Show information about Paris"

City or person?

"model" refers to car.model OR engine.model ?

"President (of the USA) before Obama?"

"composer" vs "songwriter"

"Which singer won the most Grammys?"

Grammys

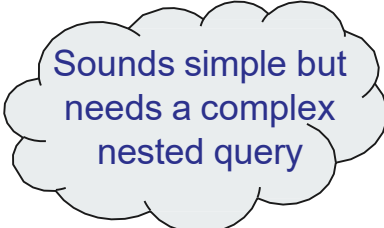
"Show most actor played movies"

??

Challenges: From the SQL side

- Complex Syntax

- SQL is a structured language with a strict grammar and limited expressivity



Sounds simple but needs a complex nested query

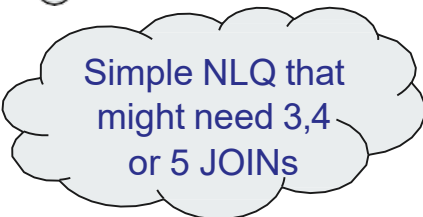


“Which countries have a GDP higher than the EU average?”

- Database Structure

- The user's data model may not match the data schema

“Find directors who released a movie this year”



Simple NLQ that might need 3,4 or 5 JOINS

Spider

- Large-scale complex and cross-domain semantic parsing and text-to-SQL dataset
 - 10,181 questions
 - 5,693 complex SQL queries
 - 200 databases from 138 different domains
- Annotated by 11 Yale students
- Queries of varying complexity
 - Categories of difficulty: Easy → Medium → Hard → Extra Hard
 - SQL elements such as JOIN, GROUP BY, UNION, INTERSECT, nested queries
- Better quality and complexity than datasets before ML

Spider: Example

Easy

What is the number of cars with more than 4 cylinders?

```
SELECT COUNT(*)  
FROM cars_data  
WHERE cylinders > 4
```

Hard

Which countries in Europe have at least 3 car manufacturers?

```
SELECT T1.country_name  
FROM countries AS T1 JOIN continents  
AS T2 ON T1.continent = T2.cont_id  
JOIN car_makers AS T3 ON  
T1.country_id = T3.country  
WHERE T2.continent = 'Europe'  
GROUP BY T1.country_name  
HAVING COUNT(*) >= 3
```

Medium

For each stadium, how many concerts are there?

```
SELECT T2.name, COUNT(*)  
FROM concert AS T1 JOIN stadium AS T2  
ON T1.stadium_id = T2.stadium_id  
GROUP BY T1.stadium_id
```

Extra Hard

What is the average life expectancy in the countries where English is not the official language?

```
SELECT AVG(life_expectancy)  
FROM country  
WHERE name NOT IN  
(SELECT T1.name  
FROM country AS T1 JOIN  
country_language AS T2  
ON T1.code = T2.country_code  
WHERE T2.language = "English"  
AND T2.is_official = "T")
```

BIRD

- BIRD (**BI**g Bench for La**R**ge-scale **D**atabase Grounded Text-to-SQL Evaluation) represents a pioneering, cross-domain dataset that examines the impact of extensive database contents on text-to-SQL parsing.
- BIRD contains over **12,751** unique question-SQL pairs, **95** big databases with a total size of **33.4 GB**.
- It also covers more than **37** professional domains, such as blockchain, hockey, healthcare and education, etc.

BIRD: Overview



BIRD: A BIG Bench for Large-Scale Database Grounded Text-to-SQLs

Large and Realistic Database Values



What is the **average salary** of the worst performing managers?

```
SELECT AVG(CAST(REPLACE(SUBSTR(T1.salary, 4), ',', '' ) AS REAL)) FROM
employee AS T1 JOIN position AS T2 ON T1.positionID = T2.positionID
WHERE T1.performance = 'Poor' AND T2.positiontitle = 'Manager'
```



Reasoned

Database:

em_id	last_name	salary
0000		US\$57,500.00
2222		US\$19,500.00
6543	Milgrom	US\$19,000.00
...	Adams	Sandy ...

Wood Emily

...

External Knowledge Reasoning



List account id who chooses **weekly issue issuance** statement?

External Knowledge:

'TOPLATEK TYDNE' stands for weekly issuance.

```
SELECT account_id FROM account WHERE account.frequency
= 'TOPLATEK TYDNE ';
```



How many accounts are **eligible for loans** in New York City?

External Knowledge:

The

condition

of loans

is that the

<https://bird-bench.github.io/>

SQL Execution Efficiency



Among the coaches who have served more than 2 NBA teams, during which coach's period of coaching, a team has the least numbers of games lost in the post-season games?

SQL₁: normal semantic parser

Run time: **22.4s**

```
SELECT coachID FROM coaches WHERE lgID='NBA' AND post_wins !=0
AND post_losses !=0 AND coachID IN
(SELECT coachID FROM coaches WHERE lgID='NBA' GROUP BY coachID
HAVING COUNT(tmID)>=2) ORDER BY post_losses ASC LIMIT 1;
```

SQL₂: **efficient** semantic parser Run time: **4.0s**

```
SELECT coachID FROM coaches WHERE lgID='NBA' AND post_wins !=0
AND post_losses !=0 AND EXISTS (SELECT 1 FROM coaches AS coaches1
WHERE (coaches1.lgID='NBA') AND (coaches.coachID=coaches1.coachID)
GROUP BY coaches1.coachID HAVING count(coaches1.tmID) >= 2
ORDER BY NULL ) ORDER BY coaches.post_losses ASC LIMIT 1;
```

How about using LLMs in Text-to-SQL?

BIRD Pipeline

Design a prompt
consists of
implementation of
ChatGPT + KG + COT

ICL Prompt Input

1-shot pseudo example

```
CREATE TABLE singer
(
  singer_id TEXT not null
    primary key,
  nation TEXT not null,
  sname TEXT null,
  dname TEXT null,
  cname TEXT null,
  age INTEGER not null,
  year INTEGER not null,
  birth_year INTEGER null,
  salary REAL null,
  city TEXT null,
  phone_number INTEGER null,
  tax REAL null,
)
```

-- External Knowledge: age = year - birth_year;
-- Using valid SQLite and understanding External Knowledge, answer the following questions for the tables provided above.

The final SQL is: Let's think step by step.

1. referring to external knowledge, we need to filter singers 'by year' - 'birth_year' > 27; 2. we should find out the singers of step 1 in which nation = 'US'; 3. use COUNT() to count how many singers. Finally the SQL is: SELECT COUNT(*) FROM singer WHERE year - birth_year > 27;

Create DDL Prompt

```
CREATE TABLE customers
(
  CustomerID INTEGER UNIQUE not null
    primary key,
  Segment TEXT null,
  Currency TEXT null,
)

...

CREATE TABLE gasstations
(
  GasStationID INTEGER UNIQUE not null
    primary key,
  ChainID INTEGER null,
  Country TEXT null,
  Segment TEXT null,
)

...

CREATE TABLE yearmonth
(
  CustomerID INTEGER not null,
  Date INTEGER not null,
  Consumption REAL null,
  primary key (CustomerID, Date),
  foreign key (CustomerID) references customers (CustomerID)
)
```

Knowledge Prompt

-- External Knowledge: August of 2012 means Date contains '201208' in the yearmonth.date of the database; Price per unit of product = Price / Amount

Task Instruction

-- Using valid SQLite and understanding External Knowledge, answer the following questions for the tables provided above.

Question Prompt

-- For all the people who paid more than 29.00 per unit of product id No.5. Give their consumption status in the August of 2012.

1-shot COT Prompt

The final SQL is: Let's think step by step.

ChatGPT COT Steps

1. referring to external knowledge, we need to calculate the price per unit of product id No.5, which is Price / Amount; 2. we should filter the people who paid more than 29.00 per unit of product id No.5; 3. we should find out the consumption status of the people in August of 2012.

SQL Output

Finally the SQL is:

```
SELECT yearmonth.Consumption FROM transactions_1k JOIN yearmonth ON transactions_1k.CustomerID = yearmonth.CustomerID AND transactions_1k.Date = yearmonth.Date WHERE transactions_1k.ProductID = 5 AND transactions_1k.Price / transactions_1k.Amount > 29.00 AND yearmonth.Date = '201208';
```

BIRD Leaderboard

Instead of simple prompts, SOTA methods rely on either strong foundation models or specially fine-tuned LLMs

What are the challenges in between?

37 Mar 17, 2023	ChatGPT + CoT HKU & DAMO [link] [Li et al. NeurIPS'23]	UNK	✓	36.64	40.08
38 Mar 17, 2023	ChatGPT Baseline	UNK	✓	37.22	39.30

	Model	Code	Size	Oracle Knowledge	Dev (%)	Test (%)
	Human Performance <i>Data Engineers + DB Students</i>			✓		92.96
1 Sep 1, 2024	AskData + GPT-4o <i>AT&T - CDO</i>		UNK	✓	72.03	72.39
2 Aug 21, 2024	OpenSearch-SQL, v2 + GPT-4o <i>Alibaba Cloud</i>		UNK	✓	69.30	72.28
3 Jul 22, 2024	Distillery + GPT-4o <i>Distyl AI Research</i> [Maamari et al. '24]		UNK	✓	67.21	71.83
4 Aug 1, 2024	ExSL + granite-34b-code <i>IBM Research AI</i>		34B	✓	67.47	70.37
5 Aug 28, 2024	Insights AI <i>Uber Freight</i>		UNK	✓	72.16	70.26
6 Aug 30, 2024	PURPLE + RED + GPT-4o <i>Fudan University + Transwarp Technology</i>		UNK	✓	68.12	70.21
7 Jul 14, 2024	RECAP + Gemini <i>Google Cloud</i>		UNK	✓	66.95	69.03
8 Jul 2, 2024	ByteBrain <i>ByteDance Infra Lab</i>		33B	✓	65.45	68.87
9 May 14, 2024	ExSL + granite-20b-code <i>IBM Research AI</i>		20B	✓	65.38	67.86
10 May 21, 2024	CHESS <i>Stanford</i> [Talaiei et al.'24]	[link]	UNK	✓	65.00	66.69

Introduction to Query Rewrite

With the rapid growth of data in various fields, it is common to take seconds or minutes, and even longer to execute an SQL query. Therefore, efficient query processing has been a crucial task in modern database systems.

How to increase the query execution efficiency effectively?

Introduction - Why Query Rewrite?

Generate a more efficient query by altering a SQL query's structure without changing the query result

```
SELECT
SUM(l_extendedprice) / 7.0 as
avg_yearly
FROM lineitem, part
WHERE
p_partkey = l_partkey AND
p_brand = 'Brand#52' AND
p_container = 'MED CAN' AND
l_quantity <
(
    SELECT 0.2 *
    AVG(l_quantity)
    FROM lineitem
    WHERE
        l_partkey = p_partkey
);
```

Rewrite



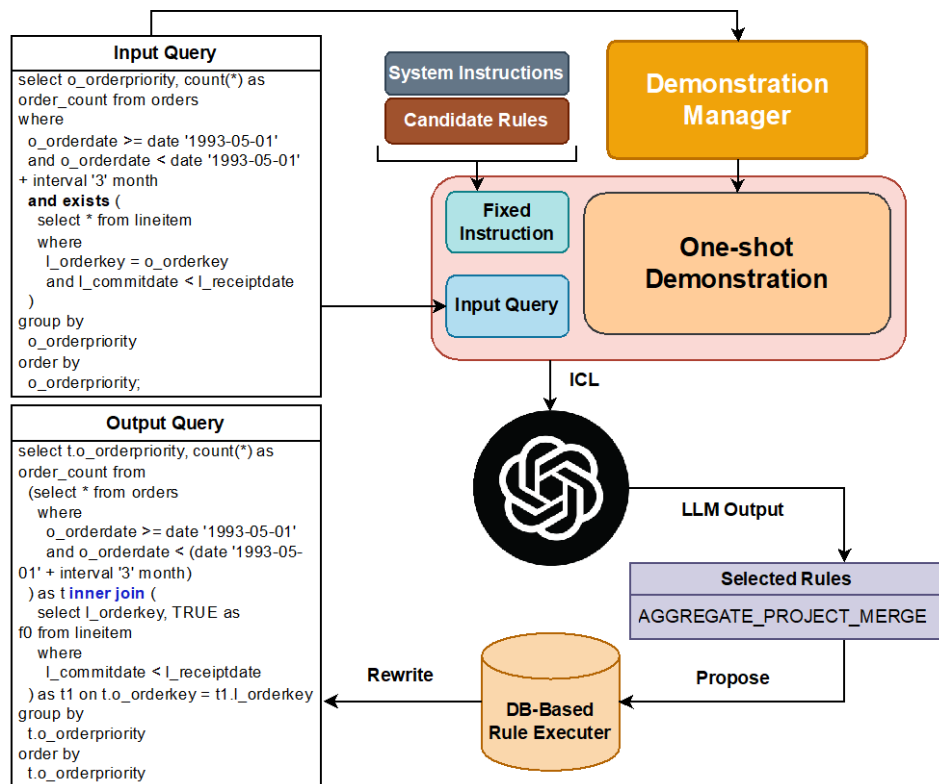
```
SELECT
SUM(lineitem.l_extendedprice) / 7.0 AS avg_yearly
FROM lineitem
INNER JOIN
(
    SELECT *
    FROM part
    WHERE
        p_brand = 'Brand#52' AND p_container = 'MED CAN'
) AS t
ON lineitem.l_partkey = t.p_partkey
INNER JOIN
(
    SELECT l_partkey, AVG(l_quantity) AS f1
    FROM lineitem
    GROUP BY l_partkey
) AS t1
ON t.p_partkey = t1.l_partkey AND lineitem.l_quantity < 0.2 *
t1.f1;
```

Original TPC-H Query, Cost = 300s

Rewritten TPC-H Query, Cost = 39s

Overall Pipeline of LLM-R²

- Our rewrite system integrates an LLM into the query rewrite system by constructing the **ICL prompt** with three main components: **Fixed instruction**, **Input query** and **One-shot demonstration**.
- Upon constructing the prompt and feeding it into the LLM, a sequence of rewrite rules is output from the LLM. These rules will undergo further processing and execution by a **database-based rule executor**.



a) Pipeline of our LLM-R System

Executing Efficiency Evaluation

- **LLM-R²** significantly reduces query execution time across the TPC-H, IMDB, and DSB datasets, outperforming all baseline methods.
- **LLM-R²** using Llama3 as backbone can also outperform both **LLM Only** and **LR**, showcasing the effectiveness of our method for both closed-source and open-source LLMs.

Execution time(sec)	TPC-H				IMDB				DSB			
Method	Mean	Median	75th	95th	Mean	Median	75th	95th	Mean	Median	75th	95th
Original	70.90	22.00	37.01	300.00	6.99	1.86	5.12	32.49	60.55	6.64	26.55	300.00
LR	39.40	22.00	32.21	159.95	6.20	1.62	4.74	32.45	59.21	5.14	53.78	300.00
LLM only (GPT-3.5)	70.67	22.00	37.01	300.00	6.96	1.86	5.10	32.49	61.60	6.53	26.40	300.00
LLM-R² (GPT-3.5)	37.23	17.40	29.80	164.12	3.91	1.33	3.52	18.16	24.11	2.16	12.61	196.61
% of Original	52.5%	79.1%	80.5%	54.7%	56.0%	71.3%	68.7%	55.9%	39.8%	32.5%	47.5%	65.5%
% of LR	94.5%	79.1%	92.5%	102.6%	63.1%	82.0%	74.3%	56.0%	40.7%	42.0%	23.4%	65.5%
% of LLM only	52.7%	79.1%	80.5%	54.7%	56.2%	71.3%	69.0%	55.9%	39.1%	33.1%	47.8%	65.5%
LLM only (Llama3)	70.80	22.00	37.01	300.00	6.98	1.86	4.99	32.49	62.02	6.62	26.55	300.00
LLM-R² (Llama3)	38.47	18.99	31.55	161.03	5.94	1.83	4.88	29.69	51.35	6.12	27.07	300.00

Table 2: Execution time v.s. different query rewrite methods

Where to find newest data system research

- Conferences
 - SIGMOD
 - PVLDB (or VLDB)
 - ICDE
 - EDBT

You are welcome to contact me if you want to do DB research.

Content of today will not appear in the final.